IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

# YENEPOYA INSTITUTE OF ARTS, COMMERCE AND SCIENCE

A Constituent Unit of Yenepoya (Deemed to be University), Balmatta, Mangalore

Department of Computer Science

# Project – High Level Design

# on

# Project Title

# Data-Driven Risk Profiling for Supply Chain

**<u>Student Name: Mohammad Fazil Anwaz</u>**

**Industry Mentor:** **Guided by:**

# 1. Introduction

**This document presents the High Level Design (HLD) for "Data-Driven Risk Profiling for Supply Chain" (PRJN26-250). It outlines system architecture, component interactions, and data flows for a weighted cyber risk scoring engine built using Python, Pandas, Matplotlib, and Seaborn in Jupyter Notebook.**

**The scoring engine enables analysts to systematically assess and rank supplier cybersecurity risk using industry frameworks including NIST CSF, ISO 27001, and CIS Controls v8.**

\

## 1.1 Scope of the Document

- **Overall system architecture and component breakdown**
- **Application design and technology stack**
- **Process flow and data flow design**
- **Data model and access mechanisms**
- **Non-functional requirements including security and performance**

## 1.2 Intended Audience

- **Students — to understand system design before implementation**
- **Faculty / Guided by — to review architectural decisions**
- **Industry Mentors — to validate design against professional standards**
- **Evaluators — to assess project design completeness**

## 1.3 System Overview

**The system takes supplier-level input scores across six cybersecurity risk dimensions, applies a user-defined weighted scoring formula, and produces ranked risk classification with visual dashboards.**

**Risk Score = $\Sigma$ ($\text{Weight}_i \times \text{Score}_i$)   for i = 1 to 6**

## 2. System Design

**The system is structured into three distinct layers:**

| Layer | Responsibility | Technology |
|---|---|---|
| Input Layer | Load and validate supplier risk data | Pandas, CSV |
| Processing Layer | Apply weighted scoring formula and classify risk | Python, NumPy |
| Output Layer | Generate visualizations and export results | Matplotlib, Seaborn |

### 2.1 Application Design

**Implemented as a Jupyter Notebook (.ipynb) with 5 modular cells:**

- Cell 1 — Library imports and environment setup
- Cell 2 — Risk factor definition, weight assignment, and dataset creation
- Cell 3 — Weighted risk score calculation and risk classification
- 
- Cell 4 — Visualization dashboard generation (bar charts, pie charts)
- Cell 5 — Results export and final summary report

## 2.2 Process Flow

**PROCESS FLOW**

**Step 1: Define Risk Factors & Assign Weights (sum = 1.0)**

↓

**Step 2: Input Supplier Scores (1-10 scale per factor)**

↓

**Step 3: Calculate Risk Score = $\Sigma$ (Weight$_i$ × Score$_i$)**

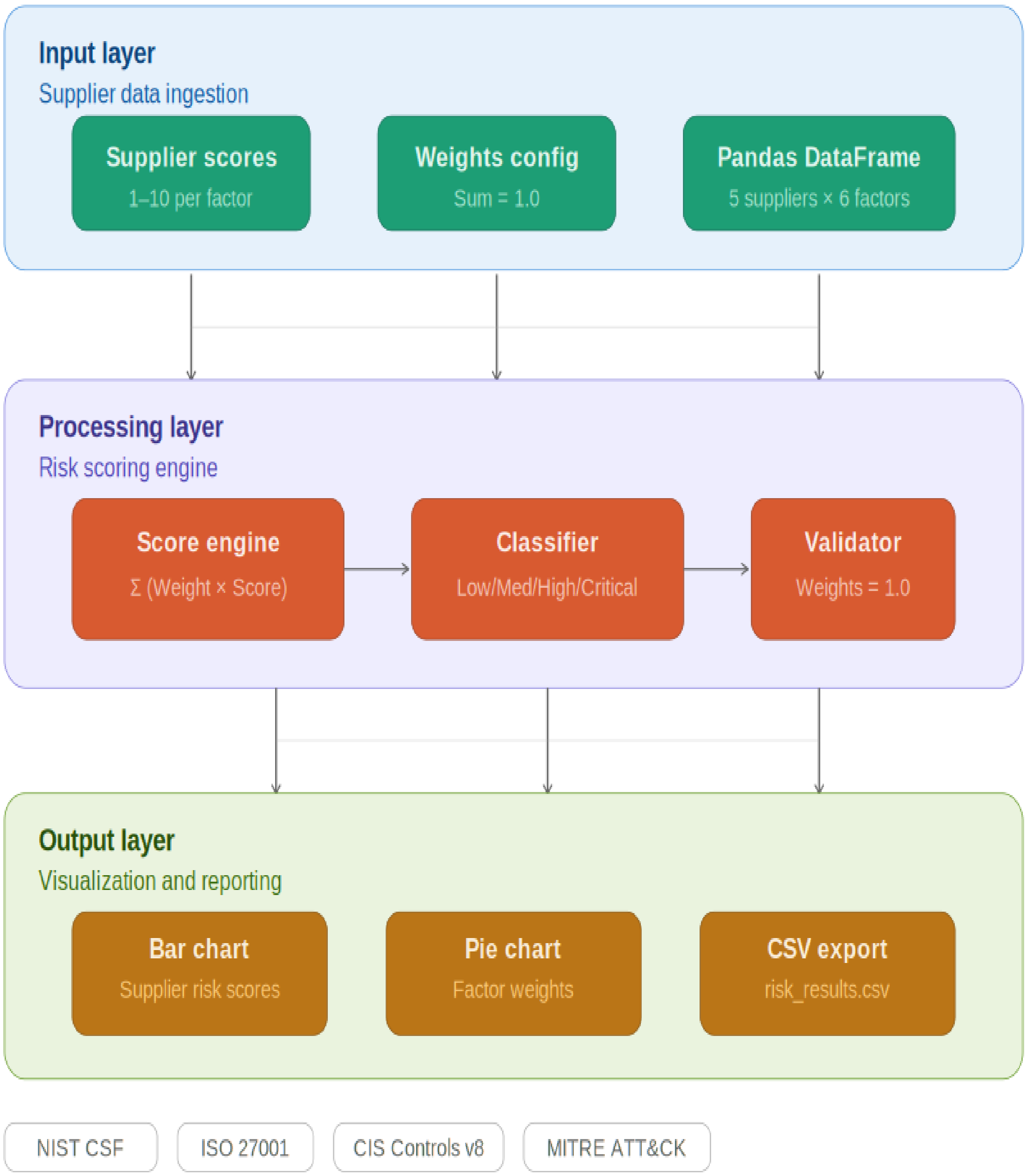
↓

**Step 4: Classify Risk — Low / Medium / High / Critical**

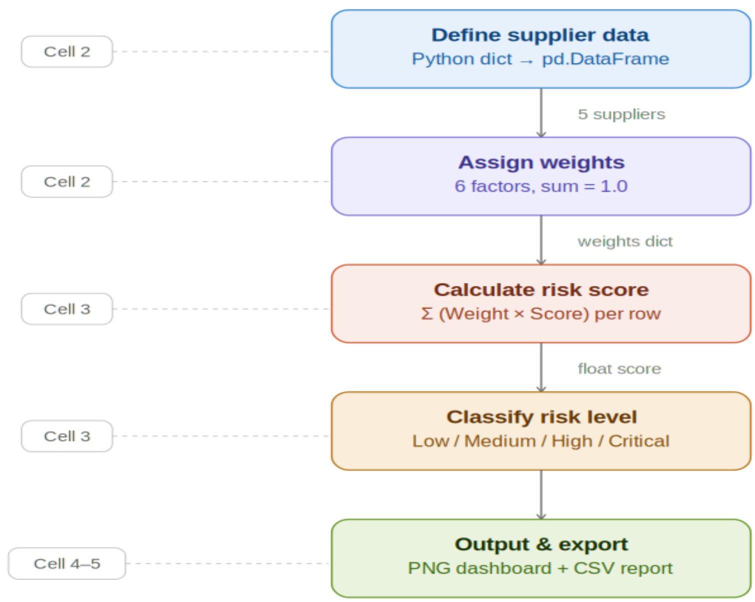
↓

**Step 5: Generate Visual Dashboard & Export CSV Report**

## Figure 1: System Architecture Diagram



## Figure 2: Data Flow Diagram




## 2.3 Information Flow

- **Raw supplier scores loaded into Pandas DataFrame**
- **Risk engine iterates over each row, applies weighted formula**
- **Calculated scores appended as new columns to DataFrame**
- **Classification module maps scores to risk tiers**
- **Output module renders charts and exports enriched DataFrame to CSV**

## 2.4 Components Design

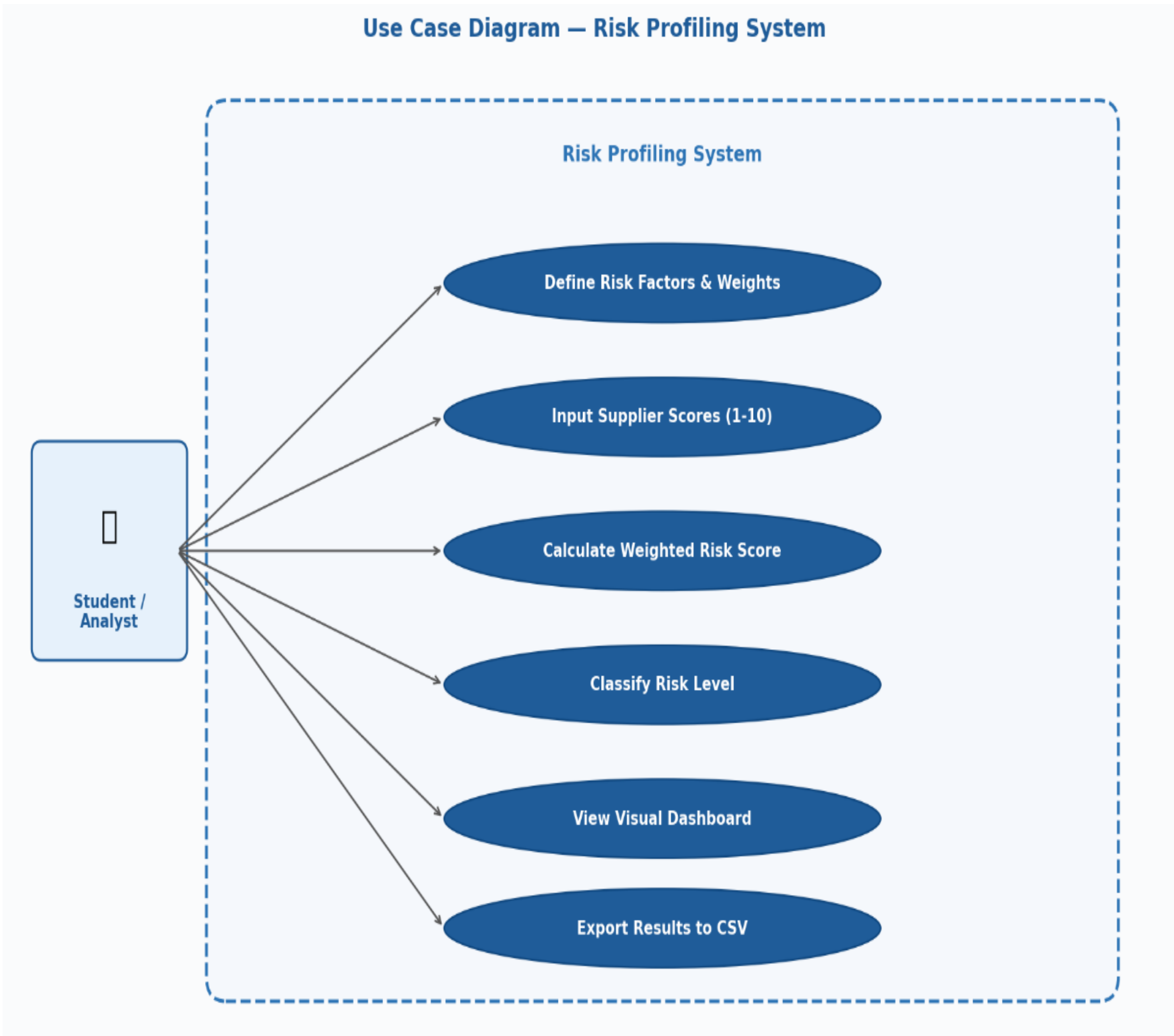
| Component | Function | Implementation |
|---|---|---|
| Data Input Module | Loads supplier data into DataFrame | pandas.DataFrame |
| Risk Calculation Engine | Applies weighted scoring formula | Python arithmetic |
| Classification Module | Maps scores to risk tiers | Python function |
| Visualization Module | Generates bar/pie charts | Matplotlib, Seaborn |
| Reporting Module | Exports results to CSV | pandas.to_csv() |

## 2.5 Key Design Considerations

- Transparency — fully interpretable weighted scoring model
- Flexibility — weights adjustable to reflect different risk priorities
- Scalability — additional suppliers or factors added without changing core logic
- Modularity — each notebook cell is independent and extendable
- Open Source — all libraries freely available with no licensing cost

## 2.6 Use Case Diagram

- **The Use Case Diagram below illustrates the interactions between the Student/Analyst actor and all system use cases within the Risk Profiling System boundary:**
-




-
- ***Figure 3: Use Case Diagram — System Use Cases and Actor Interactions***

## 2.7 Sequence Diagram

**The Sequence Diagram shows the chronological message flow between the student, Jupyter Notebook, and internal system modules during a complete risk profiling session:**

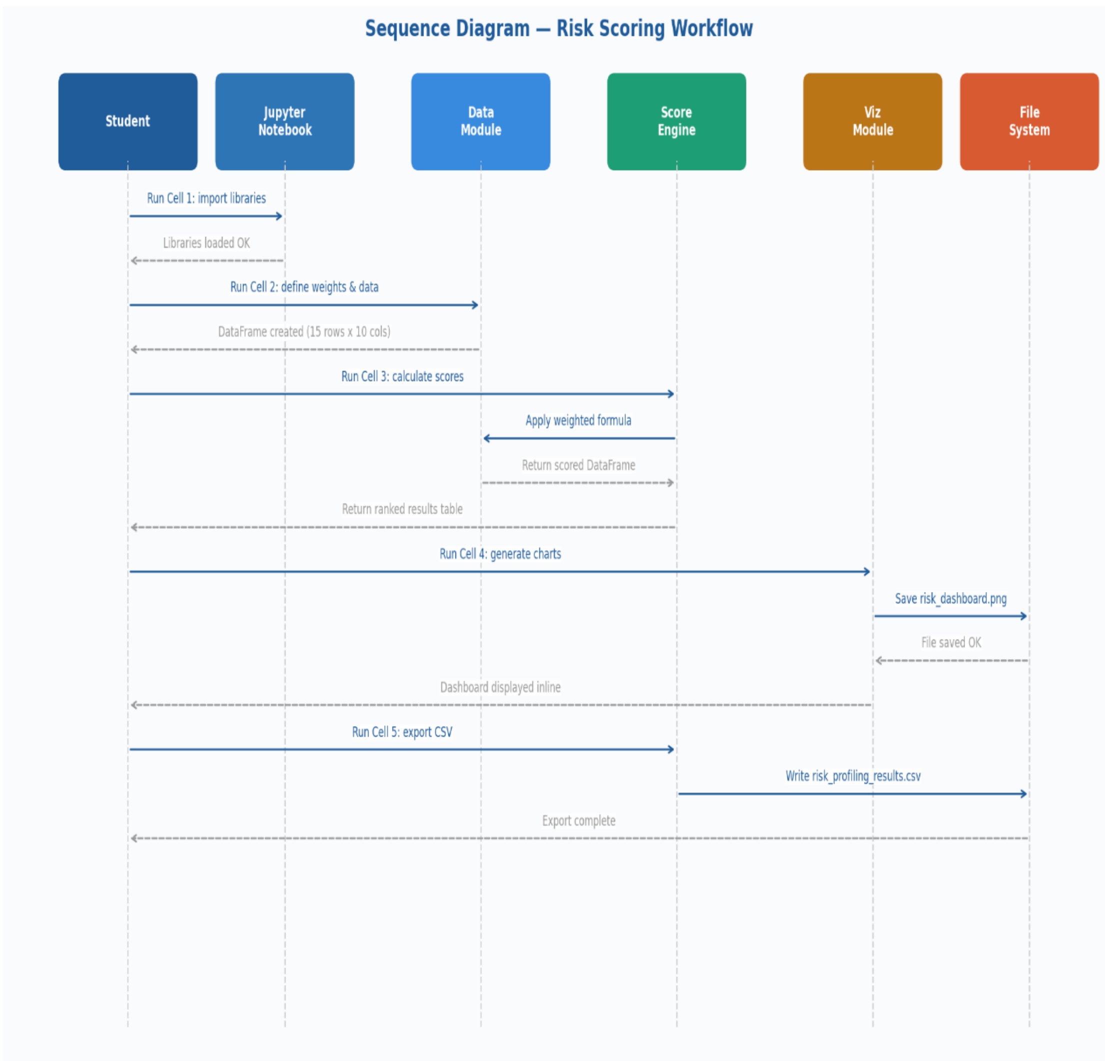



***Figure 4: Sequence Diagram — Risk Scoring Workflow Message Flow***

## 2.8 Component Diagram

**The Component Diagram illustrates all system components, their individual responsibilities, inter-component relationships, and open-source library dependencies:**

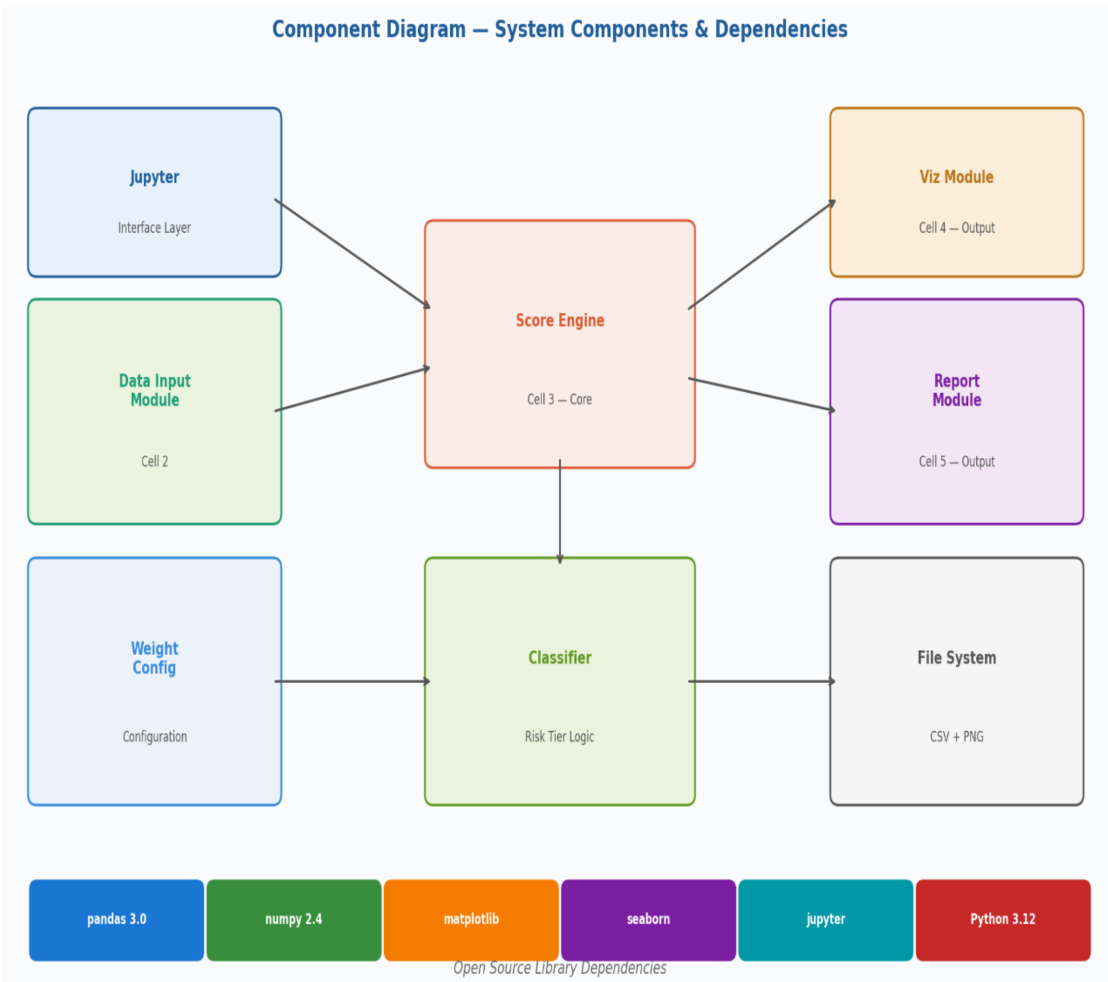



***Figure 5: Component Diagram — System Components and Dependencies***

## 2.9 Activity Flow Diagram

**The Activity Flow Diagram shows the complete step-by-step workflow including decision points for library validation and weight verification:**

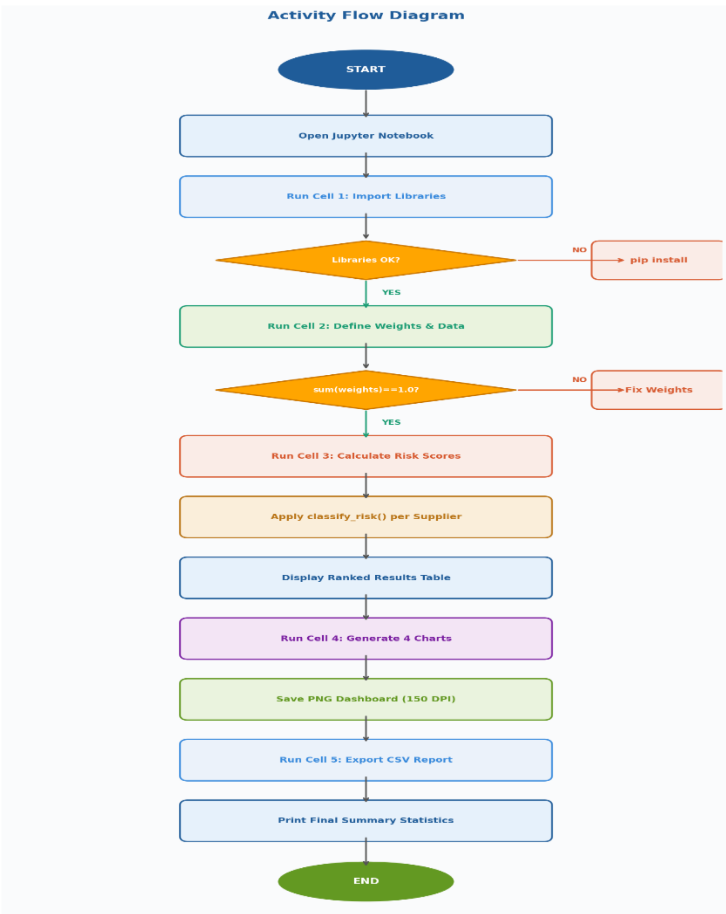



***Figure 6: Activity Flow Diagram — Complete Workflow with Decision Poin***

# 3. Data Design

## 3.1 Data Model

| Column Name | Data Type | Range | Description |
|---|---|---|---|
| Supplier | String | N/A | Name of the supplier |
| Vendor_Financial_Health | Integer | 1 – 10 | Financial stability score |
| Third_Party_Access_Control | Integer | 1 – 10 | Access control score |
| Incident_Response_Readiness | Integer | 1 – 10 | IR readiness score |
| Vulnerability_Management | Integer | 1 – 10 | Vulnerability mgmt score |
| Delivery_SLA_Performance | Integer | 1 – 10 | SLA performance score |
| Compliance_Certification | Integer | 1 – 10 | Compliance score |
| Risk_Score | Float | 0.0 – 10.0 | Calculated weighted score |
| Risk_Level | String | N/A | Low/Medium/High/Critical |

## 3.2 Data Access Mechanism

- Supplier data defined as Python dictionary, converted to Pandas DataFrame
- Factor scores accessed using column indexing: df['column_name']
- Weighted scores calculated using vectorized Pandas operations
- Final results exported using df.to_csv() for persistent storage

## 3.3 Data Retention Policies

- All data stored locally — no cloud storage used
- Results exported to risk_profiling_results.csv in project directory
- Visualizations saved as risk_dashboard.png for documentation
- No personally identifiable information (PII) stored or processed

## 4. Interfaces

| Interface | Type | Description |
|---|---|---|
| Jupyter Notebook | Web-based GUI | Primary interface for code execution |
| CSV Output | File Interface | Exported results for reporting |
| PNG Dashboard | Image Output | Visual risk dashboard chart |
| Terminal / VS Code | CLI | Used to launch Jupyter server |

# 5. Non-Functional Requirements

## 5.1 Security Aspects

- System runs entirely on local machine — no network connectivity required
- No sensitive or real supplier data used — all data is synthetic
- Output files stored locally and do not leave the user's environment

## 5.2 Performance Aspects

- Processes up to 100 suppliers instantly using vectorized Pandas operations
- Chart generation via Matplotlib completes within 2-3 seconds
- CSV export is near-instantaneous for datasets up to 10,000 rows

## 6. References


- NIST Cybersecurity Framework (CSF) v1.1 — https://www.nist.gov/cyberframework
- ISO/IEC 27001:2022 — Information Security Management Systems
- CIS Controls v8 — https://www.cisecurity.org
- MITRE ATT&CK for Supply Chain — https://attack.mitre.org
- Python 3.12 Documentation — https://docs.python.org/3.12
- Pandas Documentation — https://pandas.pydata.org/docs
- Matplotlib — https://matplotlib.org
- Jupyter Notebook — https://jupyter.org